



RoadSOS+ — Technical & Architecture Document

1. Document Overview

Project Name

RoadSOS+

Project Type

Emergency Road Assistance & Rescue Platform

Document Purpose

This document defines the technical architecture, infrastructure, system design, development stack, workflows, APIs, database structure, offline capabilities, and deployment strategy for RoadSOS+.

The goal is to create a scalable, practical, and low-latency emergency assistance platform capable of functioning even in weak network conditions.

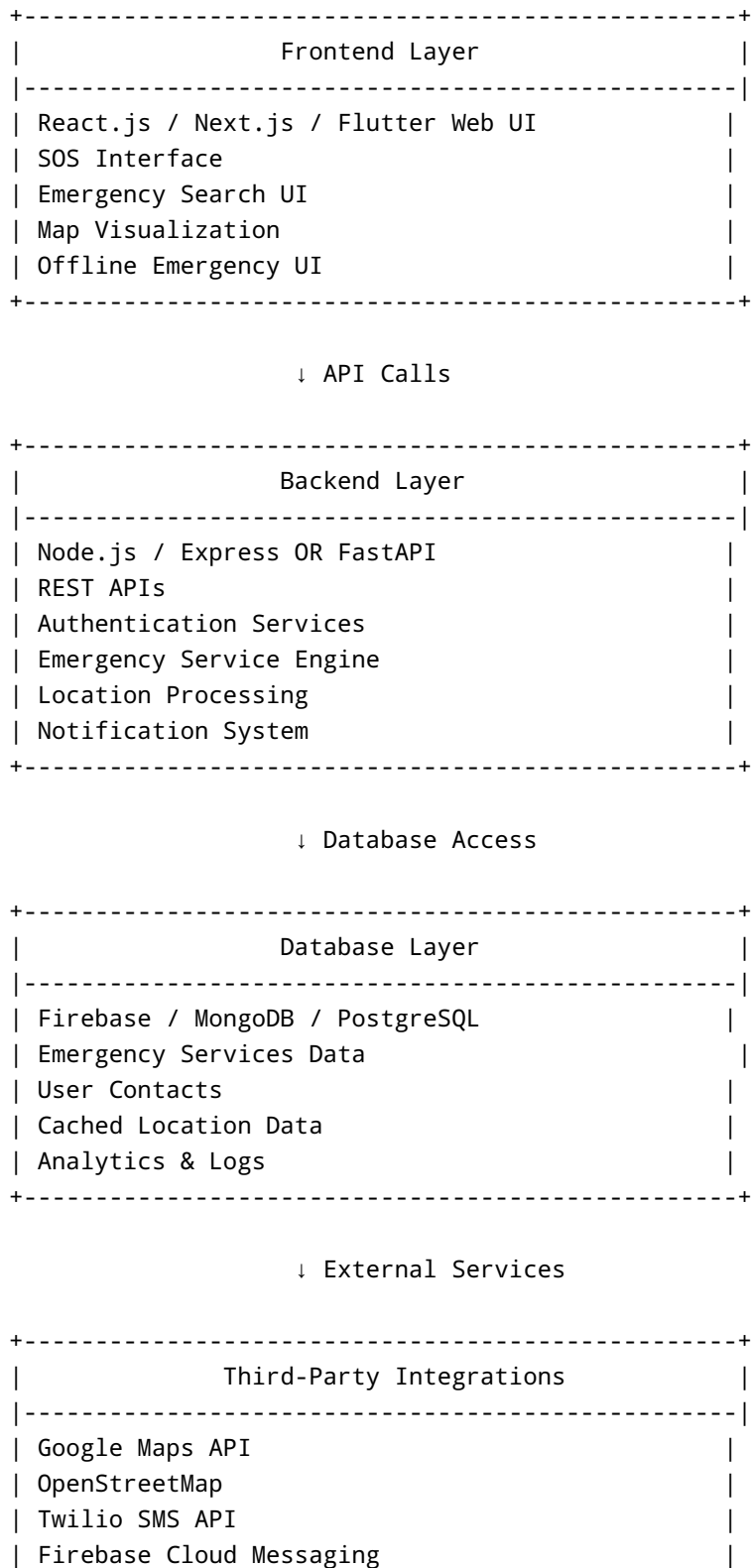
2. System Vision

RoadSOS+ is designed as a location-aware emergency response platform that helps users rapidly access nearby rescue and emergency services during road accidents.

The system emphasizes:

- Fast emergency discovery
 - Real-time location awareness
 - Offline emergency support
 - Scalable architecture
 - Lightweight performance
 - Practical deployment feasibility
-

3. High-Level System Architecture



4. Technology Stack

4.1 Frontend Technologies

Recommended

React.js + Next.js

Reasons

- Fast web development
- Component-based architecture
- Excellent PWA support
- SEO-friendly
- Easy deployment
- Strong community ecosystem

Alternative

Flutter Web

Reasons

- Cross-platform support
- Mobile-first UI
- Shared codebase for app expansion

4.2 Backend Technologies

Recommended

Node.js + Express

Reasons

- Lightweight
- Fast API handling

- Real-time friendly
 - Large ecosystem
-

Alternative

FastAPI (Python)

Reasons

- High-performance APIs
 - AI integration friendly
 - Easy asynchronous support
-

4.3 Database Technologies

Recommended

Firebase Firestore

Reasons

- Real-time database
 - Easy scalability
 - Fast prototyping
 - Serverless architecture
-

Alternative Options

- MongoDB
 - PostgreSQL
-

4.4 Maps & Geolocation

APIs

- Google Maps API
- OpenStreetMap
- Browser Geolocation API

Responsibilities

- Location capture
 - Radius search
 - Navigation launch
 - Route visualization
-

4.5 Offline Storage

Technologies

- IndexedDB
- LocalStorage
- Browser Cache API

Responsibilities

- Cache emergency contacts
 - Store last-known locations
 - Offline emergency access
-

4.6 Messaging & Notifications

Technologies

- Twilio SMS API
- Firebase Cloud Messaging

Responsibilities

- SMS fallback alerts
 - Push notifications
 - Emergency contact notifications
-

5. Functional Architecture

5.1 Emergency Discovery Module

Responsibilities

- Fetch nearby emergency services
- Calculate nearest available services
- Sort by distance and category

Input

- GPS coordinates

Output

- Nearby hospitals
 - Ambulances
 - Police stations
 - Towing services
 - Mechanics
-

5.2 SOS Management Module

Responsibilities

- Trigger emergency workflows
- Send notifications
- Share user location

Features

- One-tap SOS
 - SMS alert system
 - Live location sharing
 - Emergency contact access
-

5.3 Offline Emergency Module

Responsibilities

- Maintain usability in low connectivity
- Serve cached emergency data

Features

- Cached service lists
 - Last-known locations
 - Offline emergency screen
 - SMS-only mode
-

5.4 Location Processing Module

Responsibilities

- Fetch user GPS
- Radius calculations
- Service ranking

Features

- Browser geolocation
 - Geospatial filtering
 - Distance calculation
-

5.5 Emergency Data Management Module

Responsibilities

- Manage emergency service records
- Update contact databases
- Validate emergency data

Features

- Admin dashboard
 - Service verification
 - Regional categorization
-

6. Database Design

6.1 Users Collection

```
{
  "user_id": "USR001",
  "name": "Vaibhav",
  "phone": "+91XXXXXXXXXX",
  "emergency_contacts": [],
  "saved_locations": [],
  "created_at": "timestamp"
}
```

6.2 Emergency Services Collection

```
{
  "service_id": "HSP101",
  "service_name": "City Trauma Center",
  "service_type": "Hospital",
  "phone": "+91XXXXXXXXXX",
  "latitude": 22.5726,
  "longitude": 88.3639,
  "address": "Sample Address",
  "verified": true,
  "updated_at": "timestamp"
}
```

6.3 SOS Logs Collection

```
{
  "sos_id": "SOS1001",
  "user_id": "USR001",
  "location": {
    "lat": 22.5726,
    "lng": 88.3639
  },
  "timestamp": "timestamp",
  "status": "Triggered"
}
```

7. API Architecture

7.1 Emergency Service APIs

Get Nearby Services

```
GET /api/services/nearby
```

Parameters

```
{
  "latitude": 22.57,
  "longitude": 88.36,
  "radius": 10
}
```

Get Services by Category

```
GET /api/services/category/:type
```

7.2 SOS APIs

Trigger SOS

```
POST /api/sos/trigger
```

Request Body

```
{
  "user_id": "USR001",
  "latitude": 22.57,
  "longitude": 88.36
}
```

Send Emergency SMS

`POST /api/sos/sms`

7.3 User APIs

Save Emergency Contact

`POST /api/user/contact`

Get User Profile

`GET /api/user/profile`

8. Offline Architecture

Offline Strategy

The system uses Progressive Web Application (PWA) techniques to support emergency accessibility in low-network conditions.

Offline Features

Cached Emergency Data

- Nearby hospitals
- Police stations
- Ambulance contacts

Last-Known Location

- Stored locally

Cached Maps

- Lightweight tile caching

SMS Fallback

- Emergency alerts via SMS
-

Service Worker Responsibilities

- Cache static assets
 - Cache emergency APIs
 - Offline routing
 - Sync when internet returns
-

9. Security Architecture

Security Goals

- Protect user location data
 - Prevent unauthorized data modification
 - Secure emergency APIs
-

Security Measures

API Security

- HTTPS
- JWT authentication
- Rate limiting

Database Security

- Firestore rules

- Role-based access control

User Data Protection

- Encrypted communication
 - Secure local storage
-

10. Performance Optimization

Frontend Optimization

- Lazy loading
 - Code splitting
 - Image optimization
-

Backend Optimization

- API caching
 - Geospatial indexing
 - Optimized queries
-

Offline Optimization

- Lightweight cache
 - Reduced payload size
 - Static emergency data
-

11. Scalability Strategy

Horizontal Scaling

- Stateless backend APIs
- CDN usage
- Cloud deployment

Database Scaling

- Indexed geolocation queries
 - Distributed data storage
-

Multi-Region Expansion

- Region-based datasets
 - Country-specific emergency systems
-

12. Deployment Architecture

Frontend Deployment

Platforms

- Vercel
 - Netlify
 - Firebase Hosting
-

Backend Deployment

Platforms

- Render
 - Railway
 - AWS
 - Google Cloud Run
-

Database Hosting

Platforms

- Firebase
 - MongoDB Atlas
 - Supabase
-

13. Recommended Folder Structure

```
roadsos-plus/  
|  
├── frontend/  
│   ├── components/  
│   ├── pages/  
│   ├── services/  
│   ├── hooks/  
│   ├── utils/  
│   └── assets/  
|  
├── backend/  
│   ├── routes/  
│   ├── controllers/  
│   ├── middleware/  
│   ├── services/  
│   ├── models/  
│   └── utils/  
|  
├── database/  
|  
├── docs/  
|  
└── README.md
```

14. Recommended Development Workflow

Phase 1 — Setup

- Initialize frontend
 - Setup backend
 - Configure maps API
 - Setup database
-

Phase 2 — Core Features

- GPS integration
 - Emergency service APIs
 - SOS system
 - Nearby search
-

Phase 3 — Offline Support

- Service workers
 - Local caching
 - Offline UI
-

Phase 4 — Optimization

- Security hardening
 - Performance tuning
 - UI improvements
-

15. Future Technical Enhancements

AI Modules

- Accident-prone area prediction
 - Hazard detection
 - Vehicle diagnostics
-

Advanced Mapping

- Real-time traffic
 - Smart routing
 - Dynamic emergency rerouting
-

IoT Integration

- Vehicle sensor integration
 - Smart accident detection
-

Voice Features

- Voice SOS trigger
 - Multilingual voice assistant
-

16. Technical Risks & Mitigation

Risk	Mitigation
Weak connectivity	Offline cache + SMS fallback
Outdated emergency contacts	Periodic verification
GPS inaccuracies	Last-known location fallback
API limits	Caching + fallback providers
Slow loading	Lightweight frontend

17. MVP Technical Scope

The hackathon MVP should prioritize:

Must Have

- SOS button
- Nearby service finder
- Maps integration

- Emergency contact system
- Offline support

Nice to Have

- Hazard alerts
- Admin dashboard
- Analytics

Future Scope

- AI features
- Multi-country deployment
- Voice assistant

18. Final Technical Summary

RoadSOS+ is designed as a lightweight, scalable, and resilient emergency response platform capable of functioning in real-world accident conditions.

The architecture prioritizes:

- Fast response time
- Simplicity
- Offline reliability
- Scalability
- Practical implementation feasibility

The proposed system is highly suitable for rapid hackathon development while maintaining strong potential for future real-world deployment.
